

MFScope

India Mutual Fund Intelligence Engine — Full Project Plan

Data Pipeline · ML Scoring Engine · Dark-Mode Dashboard
Prepared August 2026

MFScope — India Mutual Fund Intelligence Engine

1. What It Does

Scrapes structured data + news for **all Indian mutual fund categories** (Large Cap, Mid Cap, Small Cap, Flexi Cap, Sectoral/Thematic incl. Defense, ELSS, Debt, Hybrid, Index), engineers features from fundamentals + technicals + news sentiment, and outputs a composite **Buy/Sell/Hold score with conviction levels** (Strong Buy → Buy → Hold → Sell → Strong Sell) per fund. Frontend is a clean **dark-mode dashboard** — not a cluttered Bloomberg-terminal look, more like a modern minimal fintech app (think Notion/Linear dark theme, not a wall of green/red numbers).

Three pieces: 1. **Data pipeline** — scrapes/pulls MF data + financial news, cleans, stores. 2. **Scoring engine** — ML model that turns raw data into a conviction-labeled score. 3. **Dashboard** — dark, minimal, fast — browse by category, drill into a fund, see the score + why.

2. Tech Stack

Layer	Choice	Why
Language	Python 3.11+ (backend/ML), TypeScript (frontend)	Your existing stack
Scraping	<code>httpx / requests</code> + <code>BeautifulSoup4</code> for static pages; <code>Playwright</code> for JS-heavy pages	Playwright handles dynamic sites (Moneycontrol, Value Research render via JS)
Official data source (priority)	AMFI India (amfiindia.com) — free daily NAV file (<code>NAVA11.txt</code>), scheme master data	This is the <i>legit, ToS-friendly, always-free</i> source — always prefer this over scraping AMCs directly
Secondary data	Moneycontrol, Value Research Online, Groww, ET Money — for expense ratio, AUM, fund manager, category ranking, historical returns	Scrape carefully — see \$5 on legality
News scraping	Economic Times Markets, Moneycontrol News, LiveMint, Business Standard RSS feeds	RSS feeds are the clean, ToS-safe way to get news — avoid raw HTML scraping where an RSS/API exists
News NLP	<code>transformers</code> (FinBERT or a multilingual finance sentiment model), or a lighter option: VADER fine-tuned/hybrid for speed	FinBERT gives you real financial sentiment, not generic positive/negative
Scheduling	<code>APScheduler</code> or a simple cron job	Daily NAV pull, hourly-ish news pull
Storage	PostgreSQL (or SQLite for MVP) for structured fund data; a vector store (optional, e.g. <code>chromadb</code>) if you want semantic news search later	Postgres scales better than SQLite once you're storing years of NAV history across thousands of schemes
Feature engineering / ML	Pandas, scikit-learn (gradient boosting: <code>XGBoost</code> / <code>LightGBM</code>) for the scoring model	Interpretable + strong tabular performance, easy to explain in a writeup
Backend API	FastAPI	Fast, typed, pairs well with a TS frontend, easy to add auth later
Frontend	React + TypeScript + Tailwind CSS + shadcn/ui (dark theme only)	Matches your existing design taste; shadcn gives you clean components without "Bloomberg terminal" density
Charts	<code>recharts</code> or <code>visx</code> on the frontend	Lightweight, themeable to dark mode
Deployment	Backend on Railway/Render, frontend on Vercel; Postgres on Supabase/Neon (free tier)	Zero-cost-friendly stack for a student project

3. Architecture



4. Fund Universe & Categorization

Cover all major AMFI-defined categories so filtering works cleanly:

- **Equity:** Large Cap, Mid Cap, Small Cap, Large & Mid Cap, Flexi Cap, Multi Cap, ELSS (tax-saving)
- **Sectoral/Thematic:** Defense, PSU, Banking & Financial Services, Pharma & Healthcare, IT/Technology, Infrastructure, Consumption, Energy
- **Debt:** Liquid, Overnight, Short Duration, Corporate Bond, Gilt
- **Hybrid:** Aggressive Hybrid, Balanced Advantage, Multi-Asset
- **Index/Passive:** Nifty 50, Sensex, Nifty Next 50, sector index funds
- **International/FoF:** US-focused, global funds (if you want breadth)

Store category as a first-class field so the dashboard can filter ("show me all Defense sector funds ranked by score").

5. Scraping Legality Notes (read before building)

- **AMFI's NAVAll.txt and scheme data are free/public** — this is your primary, safe backbone. Always prefer it.
- For AMC/aggregator sites (Moneycontrol, Value Research, Groww): check `robots.txt` for each domain, respect crawl-delay, don't hit them at high frequency, and don't scrape or redistribute content that's clearly paywalled/proprietary (e.g. Value Research's proprietary star ratings). Use them for factual data (AUM, expense ratio, fund manager name) rather than reproducing their written analysis.
- For news: RSS feeds are explicitly published for consumption — use those over scraping raw article HTML wherever a feed exists. Store headlines/summaries + link back to source; don't republish full article text.
- This matters even more than usual since it's a personal project, not a company — a scraper that respects rate limits and ToS is also just a more resilient scraper (fewer IP bans).

6. Feature Engineering (what feeds the ML model)

Performance/technical features: - Trailing returns: 1M, 3M, 6M, 1Y, 3Y, 5Y (annualized) - Rolling volatility, Sharpe ratio, Sortino ratio - Alpha/beta vs category benchmark index - Max drawdown, drawdown recovery time - NAV momentum (rate of change, moving average crossovers — reuse logic from your backtester if you build both)

Fundamental/fund-specific features: - Expense ratio (direct vs regular plan) - AUM size and AUM growth trend (rapid AUM inflow can dilute small/mid-cap fund performance — worth flagging) - Fund manager tenure, fund house track record - Portfolio turnover ratio - Category rank percentile (relative to peers, not absolute)

Sentiment/news features: - FinBERT sentiment score on recent news mentioning the fund/AMC/sector (7-day and 30-day rolling average) - News volume/attention spike detection (sudden coverage increase = event worth surfacing) - Sector-level sentiment (e.g., defense sector news sentiment feeding into all defense funds, since fund-specific news is often thin)

Macro/sector context: - Sector index performance (e.g., Nifty Defence Index for defense funds) - Category average return (to contextualize — a fund beating a falling category isn't the same as beating a rising one)

7. Scoring Model

Two reasonable approaches — pick one, or build the first and treat the second as a stretch goal:

Approach A — Rule-based composite score (build this first, it's explainable and fast to ship): - Normalize each feature to a 0-100 percentile within its category (small cap funds compared to small cap funds, not against debt funds) - Weighted sum: e.g. 40% risk-adjusted returns (Sharpe/Sortino), 20% consistency (rolling return stability), 15% cost (expense ratio, inverted), 15% news sentiment, 10% AUM/manager stability - Map composite score → label: - 80-100: **Strong Buy** - 60-79: **Buy** - 40-59: **Hold** - 20-39: **Sell** - 0-19: **Strong Sell**

Approach B — Learned model (stretch/v2): - Frame as a regression: predict forward 6-12 month risk-adjusted return using only features available *today* (careful — strict point-in-time feature construction to avoid lookahead bias, same discipline as your backtester project) - XGBoost regressor, trained on historical fund data with a proper time-based train/test split (not random split — funds are time series) - Convert predicted forward return + prediction confidence interval into the same 5-tier label - This is the version worth writing up as a small research artifact — ties nicely into the same ML rigor as your NSE anomaly detection project

Always show **feature contribution/explainability** per fund (e.g. SHAP values or just the weighted breakdown from Approach A) — "why is this Strong Buy" matters more than the label alone, especially for a portfolio piece.

8. Dashboard Design (dark mode, not Bloomberg)

Goal: dense enough to be useful, restrained enough not to look like a terminal.

- **Palette:** near-black background (#0A0B0D or similar), not pure black — single accent color for score tiers (e.g. amber/gold for Buy tiers, muted red for Sell, neutral gray for Hold) rather than a full red/green traffic-light wall
- **Typography:** one sans-serif (Inter or Space Grotesk, matches your past font choices) for UI, monospace only for numeric NAV/return figures
- **Layout:**
 - Top: category filter chips (Large Cap / Mid Cap / Small Cap / Defense / etc.) — pill-style, minimal
 - Main: card grid, one card per fund — fund name, category tag, score badge (Strong Buy/Buy/Hold/Sell), sparkline of NAV trend (small, not a full chart)
 - Fund detail view (click a card): full chart, feature breakdown, recent relevant news headlines with sentiment tags, historical score trend over time
- **No dense multi-panel terminal layout** — one focused view at a time, generous whitespace, subtle borders instead of heavy gridlines
- **Micro-interaction:** score badge could animate/pulse subtly on strong conviction scores, but keep it tasteful — this is a research tool, not a trading floor screen

9. Suggested Repo Structure

```
mfscope/
├── backend/
│   ├── ingestion/
│   │   ├── amfi_client.py
│   │   ├── news_scraper.py
│   │   └── scheduler.py
│   ├── nlp/
│   │   └── sentiment.py
│   ├── features/
│   │   └── feature_builder.py
│   ├── scoring/
│   │   ├── rule_based.py
│   │   └── ml_model.py
│   ├── api/
│   │   ├── main.py          # FastAPI
│   │   └── db/
│   │       └── models.py
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   └── styles/
│   └── tailwind.config.ts
├── notebooks/                # model dev/exploration
├── tests/
└── README.md
```

10. Build Timeline (~6-8 weeks alongside coursework)

Week	Milestone
1	AMFI NAV pipeline + scheme master data + Postgres schema; get raw fund universe loaded and categorized
2	Add secondary scrapers (expense ratio, AUM, fund manager) for one AMC first, then generalize
3	News RSS ingestion + FinBERT sentiment scoring pipeline
4	Feature engineering layer — compute all technical/fundamental/sentiment features per fund
5	Rule-based composite scoring engine (Approach A) + FastAPI endpoints
6	React dark-mode dashboard — category browse + fund detail view
7	Polish: explainability breakdown per fund, historical score trend, tests
8 (stretch)	Approach B: XGBoost learned scoring model with proper time-based validation

11. Stretch Goals

- Backtest the scoring system itself: "if you'd bought every Strong Buy label 1 year ago, what would returns have looked like vs category average?" — directly reuses your QuantEngine Backtester logic, ties the two projects together nicely.
- Alert system: notify when a fund's score crosses a tier (e.g. Hold → Buy) — good excuse to add a notification/email layer.
- Portfolio mode: let a user input their current MF holdings and get a portfolio-level score + rebalancing suggestions.
- Written report/blog post on the scoring methodology — strong addition to your research portfolio alongside the NSE anomaly detection work.

12. Immediate Next Step

Get the AMFI NAV pipeline working first (it's free, structured, and needs no scraping workarounds) — load scheme master data, tag categories, store historical NAV. That alone lets you build and test the rule-based scoring engine before you touch any scraping-fragile secondary sources.